Algebraic World Model: From Exploration to Proofs

Anonymous authors
Paper under double-blind review

Abstract

AI has made remarkable progress in mathematics, solving challenging problems with machine-checkable proofs in Lean, yet typically relying on predefined mathematical definitions and formal specifications. This raises a natural question: can it reason rigorously about an unknown world? We introduce the Algebraic World Model (AWM), a framework that turns interaction with unfamiliar environments into explicit mathematical models supporting formal reasoning. AWM explores the environment and develops executable Python hypotheses, continually testing and revising them against observations. It then structures these hypotheses into typed objects and shared rules, simplifies their structure through category-theory-inspired transformations, and formalizes the resulting model in Lean 4 to prove reachability and impossibility.

AWM enables formal proofs of impossibility beyond explored trajectories. On two impossible game levels, it constructs closed, kernel-checked Lean proofs excluding every legal winning sequence under its learned models in all six runs, while Claude Opus 5.5 and GPT-6 Astra produce none in twelve baseline runs. Four of AWM's six initial bounds match the engine optima, compared with one for Opus 5.5 and none for GPT-6 Astra; feedback-guided refinement brings AWM to six of six.

Its models make individual state transitions explicit and checkable. Across all 25 public ARC-AGI-3 games, Lean replays the winning trajectories of all 183 levels against AWM's models, and AWM with Opus 5 scores 99.9.

1 Introduction

Artificial intelligence has made remarkable progress in mathematical reasoning: automated systems now produce Lean proofs of olympiad problems and of open Erdős problems (Achim et al., 2025; Hubert et al., 2026; Sothanaphan, 2026; Tsoukalas et al., 2026). These advances invite a broader question:

Can an agent reason rigorously about an unknown world?

In formal mathematics the definitions needed to state a problem are given in advance, and a proof checker verifies whether a conclusion follows from them. In an unfamiliar environment the agent must first discover the relevant objects, infer how actions transform them and determine what counts as success; the rules required for formal reasoning are themselves hypotheses about the world, learned from experience and open to error.

Euler's treatment of the Königsberg bridges shows what such a structure can provide (Euler, 1741). Land masses become vertices and bridges become edges. A walk crossing every bridge exactly once pairs arrivals with departures at each intermediate vertex, so the graph can have at most two odd-degree vertices; the Königsberg graph has four. The graph rules out every candidate walk, including the walks nobody tried. Finding one winning route shows that a goal can be reached; showing that none exists needs a model of this kind, and in an unfamiliar environment that model has to be learned.

The Algebraic World Model (AWM) builds such a model through three stages (Figure 1). **Explore & Prototype:** the agent actively explores the environment, building and iteratively revising a provisional executable model from observed transitions. **Structure & Simplify:** it organizes this prototype into typed objects and rules, then applies category-theory-inspired transformations to discover

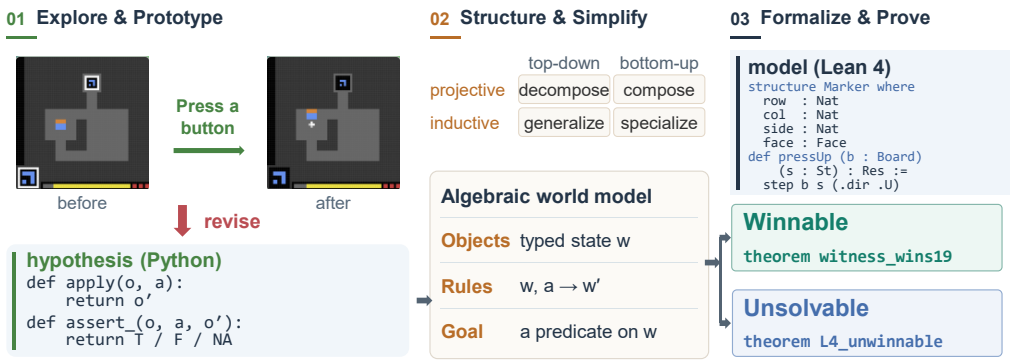


Figure 1: AWM's three-stage pipeline: interactive exploration builds a provisional executable model, algebraic transformations reveal and simplify its structure, and Lean 4 formalization enables certified proofs of reachability or impossibility.

and simplify their underlying structure while preserving empirically tested distinctions. **Formalize & Prove:** it encodes the model in Lean 4 (de Moura & Ullrich, 2021), and constructs proofs of reachability, impossibility, and other stated properties, checked by Lean's small trusted kernel. A winning witness certifies reachability; an invariant or a suitable cost lower bound, can exclude every legal winning sequence under the model.

AWM can formally prove that a task is impossible. We evaluate this ability on ARC-AGI-3 (ARC Prize Foundation, 2026b), whose games reveal neither their rules nor their goals. We construct two impossible variants of `sc25` levels by preserving their mechanics and setting action budgets just below the minimum winning costs (§4). Across six runs, AWM constructs closed, kernel-checked Lean proofs excluding all legal winning sequences under its learned models, while Claude Opus 5.5 and GPT-6 Astra produce no such proofs in twelve baseline runs.

The difference extends beyond proof construction. On one level, Astra correctly judges impossibility but incorrectly estimates the minimum winning cost as 22, whereas an independently rebuilt AWM model proves the engine-optimal bound of 19. More broadly, AWM's initial bounds match the engine optima in four of six runs, compared with one for Opus 5.5 and none for Astra; generic discrepancy feedback brings AWM to six of six, each supported by a new closed proof.

AWM's models can be understood and checked step by step. On all 25 public ARC-AGI-3 games, AWM builds models with a median of six object types and ten rules, and Lean replays the winning run of every one of the 183 levels against them (§3); with Opus 5 it still clears the games at the level of the best code-executing systems (99.9).

We make three contributions.

- **Algebraizing an unknown world.** A three-stage method that transforms interaction into executable hypotheses, structures them into compact algebraic world models through category-theory-inspired transformations, and formalizes them in Lean 4 for machine-checked reasoning (§2). On lp85, the shared rule it finds shortens proofs of unsolvable instances by two to three times in lemmas (§2.5).
- **Knowing when a task is impossible.** AWM can formally prove that no legal sequence reaches a goal, a conclusion no amount of play establishes and agents without AWM do not reach. On two sealed levels, all six AWM runs prove impossibility in Lean under their learned models, while Claude Opus 5.5 and GPT-6 Astra produce no such proof in twelve runs, and with the same base model and budget AWM reaches a verdict in 9 of 10 runs, against 4 of 10 without it (§4).
- **Understandable models, grounded at every step.** Every rule of an AWM is tested against the record, every simplification keeps the distinctions already tested, and every conclusion is a theorem that names the rules it rests on. These named rules locate an injected error in 29 of 42 runs, against 16 without them (§4.3). On all 25 public ARC-AGI-3 games, Lean checks that these models reproduce the winning runs of all 183 levels, and with Opus 5 AWM still clears the games at the level of the best code-executing systems (99.9; §3).

2 METHOD

AWM aims to turn interaction with an unknown environment into a mathematical model that supports rigorous reasoning about both reachability and impossibility. This requires more than collecting rules that explain observed transitions: the rules must form a coherent representation of the environment, and that representation must support reasoning about action sequences the agent has never explored.

We first describe the target model (§2.1), then present the three stages that construct and use it (Figure 1): Explore & Prototype develops executable hypotheses through interaction; Structure & Simplify organizes them into a compact algebraic model; and Formalize & Prove translates the model into Lean 4 and establishes machine-checked results.

2.1 THE MODEL: OBJECTS, RULES, AND FORMAL REPRESENTATION

An algebraic world model represents an environment through typed objects, rules, and a goal predicate, with executable Python and formal Lean 4 representations supporting empirical testing and rigorous reasoning.

Objects. An object type specifies structured data and its validity conditions; an object is a value of that type. A parser maps observations to typed states w, including history or auxiliary information when the current observation alone is insufficient.

Rules. A rule specifies when a mechanism applies and how it behaves. Its Python representation defines an applicability scope and a checker that reports agreement, contradiction, or non-applicability on a transition; effect rules also predict all or part of the successor state. Together, the rules specify legal actions, their effects and costs $\text{cost}(w, a)$, while the goal predicate identifies successful states.

When generalization merges several rules into one, the shared rule $r(\theta)$ takes a parameter θ read from the state; a rule for a single mechanism needs none. This structure allows common properties to be expressed and proved across multiple instances rather than separately for each rule.

Formal representation. The same model is also represented in Lean 4, where objects and rules become formal definitions and properties of their dynamics become theorems. Python supports empirical testing against observations, while Lean supports kernel-checked reasoning about legal action sequences under the model. The formal representation is constructed in §2.4.

2.2 EXPLORE & PROTOTYPE

Setting. We consider deterministic, goal-directed environments with partial observations, modeled in the POMDP setting (Kaelbling et al., 1998). At step t, the agent observes o_t, takes an action a_t, and receives o_{t+1} without access to the underlying state or transition function. Its experience forms a corpus $\mathcal{D}_t$ of distinct transitions $x = (o, a, o')$.

Building and testing hypotheses. AWM actively explores the environment and maintains a provisional executable model as a *concept book*. Each concept is a Python hypothesis about an environmental mechanism, containing a natural-language claim, an applicability scope, supporting transitions, a checker, and an optional predictor.

For a concept c, its checker returns $v_c(x)$: `TRUE` if the transition agrees with the hypothesis, `FALSE` if it contradicts it, and `NA` if it falls outside its scope. The optional predictor specifies expected action effects for comparison with subsequent observations.

After each action batch, AWM rechecks the concept book $\mathcal{C}_t$ against the expanded corpus $\mathcal{D}_t$. Further interaction is allowed only when

$$\forall c \in \mathcal{C}_t,\ \forall x \in \mathcal{D}_t : \quad v_c(x) \notin \{\mathsf{FALSE}, \mathsf{ERR}\}, \tag{1}$$

where ERR denotes checker failure, including exceptions and timeouts.

This recheck ensures that contradicted or failing concepts must be revised or withdrawn before exploration continues. The agent then keeps revises and uses its hypotheses to plan toward inferred goals or probe competing explanations.

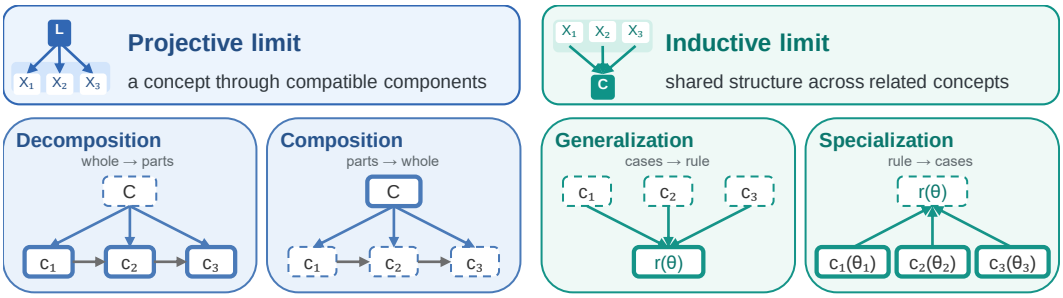


Figure 2: The four abstraction transformations, grouped by the limit they follow; each applies to object types and to rules. Arrows follow the limit; a dashed box is removed by the transformation and a bold box is created. A parameter θ appears only on a shared rule $r(\theta)$; in specialization, $c_i(\theta_i)$ is that rule with its parameter fixed to θ_i.

The resulting prototype captures tested mechanisms but may describe related behaviors independently, motivating the next stage.

2.3 Structure & Simplify

The concept book may contain fragmented, overlapping descriptions of environmental mechanisms. Structure & Simplify turns these hypotheses into a compact model by identifying shared structure and reorganizing components through their relationships.

AWM first extracts the required object types, constructs a parser from observations to typed states, and rewrites concepts as rules over these objects. It then simplifies the resulting *components*—object types and rules—through four transformations inspired by category theory. In category theory, a *projective limit* assembles compatible descriptions into a coherent whole, while an *inductive limit* glues objects along shared structure (Mac Lane, 1998; Kashiwara & Schapira, 2006; Yuan, 2023). Informally, a projective limit describes a whole through its compatible parts, as a desk is described by its top, drawers and legs, and an inductive limit describes what several things share, as "animal" covers lion, tiger and elephant. These dual perspectives guide four transformations (Figure 2).

Following projective limits, *decomposition* expresses a complex component through simpler components and their compatibility relations. Conversely, *composition* identifies components determined by compatible existing ones, allowing redundant definitions to be removed.

Following inductive limits, *generalization* extracts shared structure from multiple components, expressing source rules c_i through a parameterized operation $r(\theta_i)$ with state-dependent θ_i. Conversely, *specialization* refines overly general components into more specific cases or introduces parameters to reduce exceptions and redundancy.

Preserving tested distinctions. Every transformation must preserve verdicts on recorded transitions. For rules, AWM also evaluates a diagnostic set $\mathcal{N}$ of perturbed recorded transitions, most of them invalid under the environment's dynamics, to prevent simplification from losing previously detected rejections.

Define $\text{Rej}(c) = \{x \in \mathcal{N} : v_c(x) = \textsf{FALSE}\}$ and $\text{Rej}(S) = \bigcup_{c \in S} \text{Rej}(c)$. Replacing rules S with T must preserve coverage of recorded transitions and satisfy

$$\text{Rej}(T) \supseteq \text{Rej}(S). \tag{2}$$

For object types, which have no checkers, transformations are evaluated by distinctions between recorded states, using state pairs as witnesses, and by rule verdicts once rules are available.

AWM applies these transformations and tests to object types, then rules, and finally rechecks object types against the settled rules. The resulting compact structure provides a common representation for formal reasoning. The experiments reported here use the version of this procedure that applies generalization and deletion; the full set of four transformations is used in the released code.

2.4 Formalize & Prove

AWM completes the model in Lean 4, encoding object types as structures or inductive types, rules as `def` definitions, and derived properties as `theorem` statements. The transition function dispatches to named rules, while unresolved assumptions remain explicit theorem hypotheses rather than unchecked `axiom` declarations. Replayed along the recorded winning runs, this compact algebraic model reproduces the observed behavior on every component it models, so its theorems concern the same dynamics the agent played, now in a form where claims about every action sequence can be proved.

Proving reachability and impossibility. A level is *winnable* in the model if a legal action sequence reaches the goal. Lean proves reachability by checking such a witness. To prove impossibility, AWM instead rules out all legal winning sequences, including unexplored ones, using an invariant that excludes the goal or a cost lower bound. For the latter, a potential V, a lower estimate of the cost still needed from each state, satisfies

$$V(w) \le \mathrm{cost}(w,a) + V(w') \quad \text{for every legal transition,} \qquad V \le 0 \quad \text{on the goal,} \tag{3}$$

so every winning sequence costs at least $V(w_0)$, proving impossibility when this exceeds the budget. All proofs are closed and checked by the Lean kernel, with no unproved placeholders (`sorry`) and no steps that bypass the kernel (`native_decide`), establishing rigorous guarantees under the learned model's explicit premises.

2.5 Example: one rule for four mechanisms

In game `lp85`, buttons move colored tiles, and a level is won when every slot holds its matching color. During Explore & Prototype, the agent discovers eleven concepts describing four button mechanisms: belts, stars, hubs, and panels (Figure 3a). In Structure & Simplify, AWM applies the generalization transformation to recognize that all four mechanisms permute tile positions, replacing the eleven concepts with a shared operation, `press_permutes`, parameterized by the pressed button. The unified rule loses none of the 55 invalid transitions its eleven sources rejected and catches 21 more. On 34 held-out instances, the merged model is proved in every attempt, and its proofs of unsolvable instances use two to three times fewer lemmas and up to 2.7 times fewer lines than proofs over the eleven separate concepts.

In Formalize & Prove, Lean certifies a five-click winning sequence (Figure 3b) and proves a recolored variant unwinnable using the color-count invariant shared by all button permutations (Figure 3c). The final model contains six object types and six rules, and Lean replays all 199 recorded transitions without mismatch.

3 Twenty-five public games

ARC-AGI-3 (ARC Prize Foundation, 2026b) scores each level by the squared ratio of a human baseline's action count to the agent's, weights later levels more, and gives an uncleared level zero. We evaluate AWM with Claude Opus 5 on all 25 public games (183 levels), starting each game with an empty concept book.

Clearing the games. AWM with Opus 5 achieves a mean score of 99.9 on the 25 games, matching leading code-executing systems (Table 1), so building an explicit model does not hurt its play.

Compact models that reproduce observed behavior. Across the 25 games, AWM writes 742 executable concepts, each checked against every recorded transition, and withdraws the 143 that a later observation refutes. Structure & Simplify organizes the rest into 170 object types and 283 rules, a median of 6 types and 10 rules per game. Shared patterns recur across games, including translations, permutations and unified goal conditions. Lean replays the winning trajectory of every one of the 183 levels from its initial state and verifies its final state against the modeled goal (Table 2).

Generalization beyond observed transitions. Agreement with recorded play alone does not establish whether learned rules capture unseen behavior. We therefore freeze the models of `lp85` and

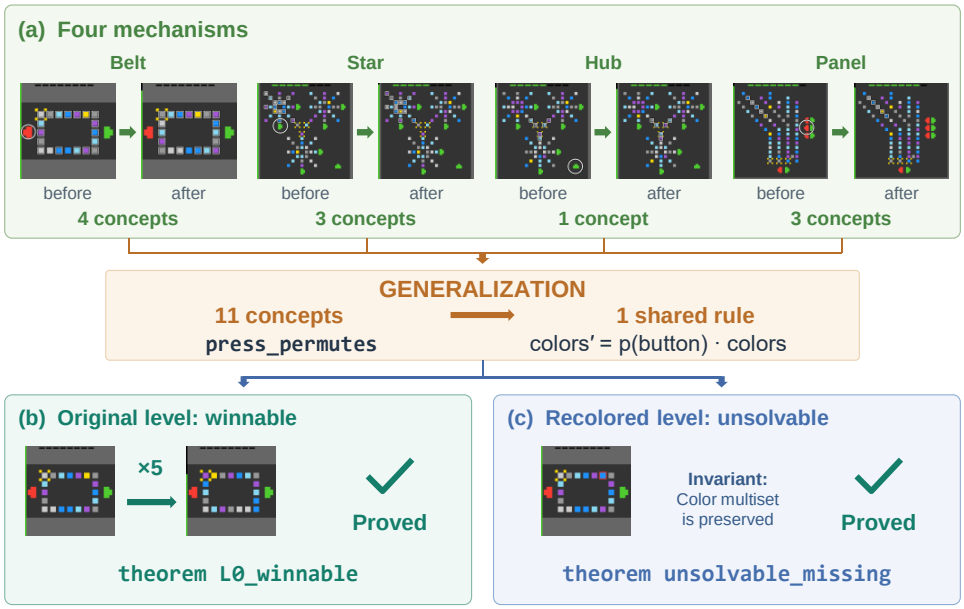


Figure 3: From shared structure to formal proofs in `lp85`. (a) Eleven concepts describing four button mechanisms are generalized into a shared permutation rule. Each before/after pair shows a recorded press, with the pressed button circled and changed tiles outlined. (b) A five-click sequence witnesses reachability. (c) Recoloring the only yellow tile makes the goal unreachable, since every press preserves color counts. Both conclusions are formally proved in Lean 4.

Table 1: Official scores on the 25 public ARC-AGI-3 games. In `su15` and `sp80` the agent was also given mechanisms from the games' public source code.

System	Mean score
GPT-6 Astra (adapter) (ARC Prize Foundation, 2026a)	100.0
baseline1 (Rodionov, 2026b)	99.0
Memento 3 (Zhao et al., 2026)	100.0
AWM (ours)	99.9

Table 2: Formalization and verification of the 25 learned world models in Lean 4. Axioms are those Lean's own logic uses; no proof adds axioms of its own.

Levels replayed	Rules formalized in Lean	Theorems	Axioms
183 / 183	220	1,374	only `propext`, `Classical.choice`, `Quot.sound`

`ft09` after exploration and evaluate them on transitions from random play with previously unseen screens (seeds 0–9; detailed results in the supplementary material). On new trajectories the frozen rules accept every valid transition; on `lp85` layouts never seen during play they predict every next frame (4,160 of 4,160); on `ft09` they reject every corrupted transition and every transition of the changed mechanic (4,031 of 4,031).

The changed mechanic modifies `ft09`'s hidden click pattern, causing each click to change 110 or 146 cells instead of 38; rejecting these transitions shows that the rules detect behavior inconsistent with the learned dynamics. §4 examines whether such models support proofs of impossibility.

Table 3: Impossibility reasoning across six runs per agent. Bound accuracy is evaluated separately against engine optima, before/after feedback-guided repair. Claimed by attempt 3 counts runs whose first impossibility claim comes by attempt 3. All agents receive the same task, interface and attempt budget; the repair round gives baselines the same feedback and attempts as AWM. One level-4 run, stopped after attempt 3 under a protocol with an element prior, replaces a run whose Lean check did not finish; its Lean proof was rebuilt and checked independently (Appendix B.6).

Agent	Kernel-checked model proofs	Correct bounds (§4.2) initial → feedback	Ever claimed impossible	Claimed by attempt 3
Claude Opus 5.5	0/6	1/6 → 1/6	3/6	0/6
GPT-6 Astra	0/6	0/6 → 2/6	2/6	0/6
AWM	**6/6**	**4/6 → 6/6**	**6/6**	**6/6**

4 Proving impossibility

A winning trajectory establishes reachability; impossibility requires excluding every legal winning sequence within the budget, including those never explored. We test whether AWM can turn learned rules into such proofs, measuring model-relative proof validity apart from agreement with the game.

4.1 The game, the seal, and the baselines

In `sc25`, a block moves on a grid and casts spells through a 3×3 panel to reach the exit within the level's budget. Moves and panel clicks cost one unit, completing a spell one more, fuel refunds ten, and the winning move is free. We build two levels from levels 4 and 5 by rearranging selected elements, keeping the game's mechanics, and seal them at budgets of 18 and 35. Exhaustive engine search over all four openings and all 13 actions puts their cheapest wins at 19 and 36, so neither sealed level can be won, while one more unit would permit a win.

Claude Opus 5.5 and GPT-6 Astra receive the game interface and coding tools, including simulator construction and search; AWM uses Opus 5.5 with the procedure of §2 and a generic Lean skeleton. Each agent plays three runs per level, with up to eight attempts per run; an attempt ends at game over or abandonment. After each attempt, every agent is asked whether the sealed level can be won and to justify its verdict; AWM justifies an impossibility verdict with a Lean proof under its learned model. An impossibility verdict is justified by a lower bound on the cheapest win that exceeds the budget. We judge separately whether the justification holds under the agent's learned model and whether its bound equals the engine's optimum, 19 or 36.

4.2 Certified lower bounds

Proving beyond explored trajectories. All six AWM runs yield closed, kernel-checked impossibility proofs under their learned models and explicit premises; none of the twelve runs without AWM produces such a certificate (Table 3). The run's theorem `L4_unwinnable`, for example, states that no action list of its model wins within the budget of 18, and `witness_wins19` gives one that wins at 19. AWM establishes such bounds through potential certificates (Eq. (3)) or closure certificates, which give a set of states that every action keeps the play inside and that contains no goal, over arbitrary legal action lists. External search may propose certificate data, but the resulting optimality claim is verified in Lean rather than trusted from the search. In a mutation test on three models, Lean rejects every faulty certificate that a Python checker rejects, and also catches every vacuous premise (6 of 6), reversed bound (3 of 3) and drift between the Python and Lean versions of a model (3 of 3), all of which the checker accepts (Appendix C). The proofs explicitly expose their modeled rules and premises, allowing proof validity to be assessed separately from agreement with the game.

Beyond unproved numerical judgments. Only five of twelve baseline runs claim impossibility, four with incorrect bounds. Even the sole correct Claude Opus 5.5 bound lists "that my hand-counted path lengths are truly minimal" as *assumed, not checked*. The missing step is a global lower bound: checking a route does not exclude cheaper alternatives. AWM discharges this obligation through kernel-checked proofs under its learned models. In a comparison with the same task, base model

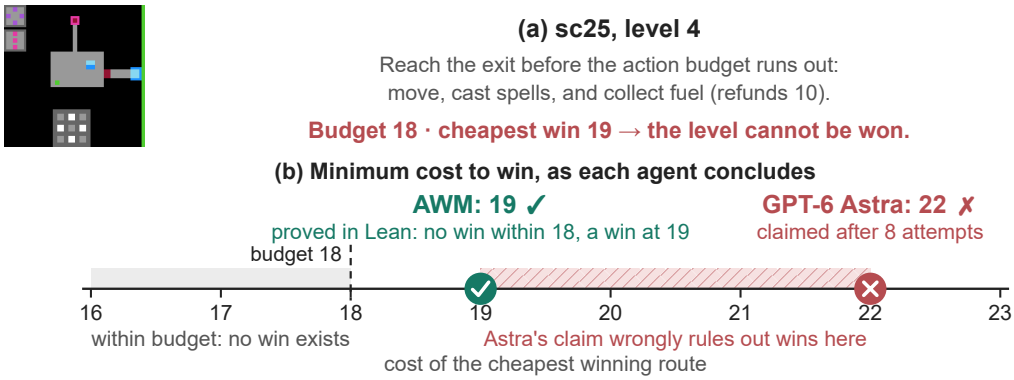


Figure 4: Sealed sc25 level 4: the budget is 18 and the cheapest win costs 19, so the level cannot be won. GPT-6 Astra concludes that a win needs at least 22, which would also rule out wins at 19 to 21; AWM proves in Lean that no route wins within 18 and gives a route that wins at 19.

and budget, AWM, which also receives the prior that the rules stay fixed, reaches a verdict in 9 of 10 runs, against 4 of 10 for the same agent without it (Appendix B.11).

Proof validity and bound accuracy are distinct. Four of six AWM bounds agree with the engine optima before feedback, compared with one for Opus 5.5 and none for GPT-6 Astra. We count a bound as correct if it matches the engine optimum or differs only by charging the winning move, which the engine waives and which cannot be observed on a sealed level; this is a cost-accounting convention, not a numerical tolerance. The two incorrect AWM bounds, 20 on L4 and 47 on L5, come from learned rules stricter than the game; because each proof names the rules it rests on, the error can be traced to a rule and repaired (§4.3). Figure 4 contrasts Astra's incorrect estimate with a tight, formally proved model bound.

4.3 Auditing premises and proving again

After the initial evaluation, the two AWM runs with bounds of 20 and 47 receive a generic signal that the engine permits a cheaper win, without the correct bound, route or faulty rule being named. AWM relaxes its explicit rules, tests the resulting disagreements through interaction and proves the refined conclusions again.

On L4, growing onto fuel is found to collect it, reducing the bound from 20 to 19. On L5, a shot's failure is traced to terrain rather than limited range, reducing 47 to 37 with the winning move charged (36 otherwise). Both repairs finish within four additional attempts, and each ends with a new closed proof. With additional feedback, AWM reaches correct bounds in all six runs.

We provide the same discrepancy feedback and additional attempt budget to the four baseline runs with incorrect bounds. Astra corrects both of its bounds, but neither produces a formal proof, while Claude Opus 5.5 fails to correct either.

The proof points to the error. To test whether the rules a proof names suffice to find an error without being told where it is, we inject single-rule errors into models that match the engine exactly and give agents only the proof's rule dependencies. On level 4 they locate the error in 29 of 42 runs, nearly as often as agents told which rule is faulty (30) and almost twice as often as agents given per-rule evidence alone (16; paired difference 0.24, 95% CI [0.08, 0.40]). They reach it within a median of 27 actions, where most evidence-only agents do not reach it within the 50-action budget, and they raise no false alarm on error-free models, against 0.33 per run for the evidence-only agents. On a faulty model from one of AWM's own runs, every run finds the error within 15 actions (Appendix B.10). Repaired proofs can also hold for the game itself: after its refuted transitions are replaced by observed ones, a level-5 bound holds against every engine transition within the cap, 17,194,487 in all (Appendix B.8).

5 Related work

Executable world models. Code world models are learned from interaction for planning (Tang et al., 2024), refined by generation, testing and repair (Dainese et al., 2024) or composed from programmatic experts (Piriyakulkij et al., 2025); theory-based agents explore and plan with learned theories (Tsividis et al., 2021), and TheoryCoder-2 learns reusable abstractions for hierarchical planning (Ahmed et al., 2026). Such models are central to recent ARC-AGI-3 agents (Rodionov, 2026a; Courtis et al., 2026; Rodionov, 2026b): Twin does not act until its simulator reproduces every observed transition (Skoutnev et al., 2026), and Tycho finds that a closer match does not by itself yield better play (Lehmann et al., 2026). The closest work, Memento 3, compiles a natural-language rulebook into code and accepts an update only if an LLM judges it faithful and replay is exact (Zhao et al., 2026). Yet a code world model can pass a sampled-transition check at 100% accuracy and still lose because it misses a key rule (Aguilar Martín, 2026). AWM instead abstracts its rules into a shared structure, tests them beyond the record, and proves model-relative reachability and impossibility.

Certified impossibility. Unsolvability certificates let an independent checker confirm that a planning task has no plan (Eriksson et al., 2017), in the spirit of certifying algorithms (McConnell et al., 2011), and certificate checking has been formally verified for temporal planning (Wang & Abdulaziz, 2025); all start from a supplied model. Impossibility transfers from an abstraction only if it over-approximates the real system, budget semantics included (Clarke et al., 1994), and action-model learning gives conditions under which learned models do so (Aineto & Scala, 2024). AWM learns its model from interaction; Lean checks deductions relative to that model, not the truth of its rules, which stay open to testing.

Formal reasoning. Neural provers produce Lean proofs at olympiad level (Achim et al., 2025; Chen et al., 2025; Hubert et al., 2026; Ren et al., 2025), AI systems have produced Lean proofs of open Erdős problems (Sothanaphan, 2026; Tsoukalas et al., 2026), and autoformalization translates informal statements into formal ones (Wu et al., 2022). There the objects of proof are given mathematics; AWM must first infer the dynamics and costs its proofs quantify over, which makes the fidelity of its formal model an empirical question.

6 Limitations and conclusion

Limitations. Our current framework relies on ARC-like environments with discrete actions and immediate, observable feedback, where learned rules can be checked against recorded transitions. Extending this process to real-world settings with partial observations, delayed feedback, or stochastic dynamics is the next step. Lean guarantees the correctness of proofs under the learned model's explicit premises, and whether these premises capture the environment is tested by interaction: the explicit rules and kernel-checked proofs separate errors in the learned model from errors in reasoning, so a discrepancy can be traced to its assumption and addressed by further exploration. Broader environments and tasks are the natural next test of such formal reasoning.

Conclusion. Formal reasoning traditionally begins with predefined objects and rules; AWM discovers them through interaction, structures and simplifies them through category-theory-inspired transformations, and formalizes the result in Lean, where reachability and impossibility over action sequences never explored become kernel-checked theorems. On sealed levels, AWM proves in Lean what agents without it can only guess, and when such a proof is wrong, the rules it names locate the error in 29 of 42 runs.

Reproducibility statement

The readings of §2 and §3 come from the benchmark repository at commit `fa928185` (built 2026-10-24). Each game's model, tests and Lean project are regenerated by one command per game; the per-game Lean projects are checked with Lean 4.34.0 and do not use Mathlib. The readings of §4 come from the archive of the budget-sealed instance: both engine versions with their md5, the abstract model and its cross-check scripts with their raw outputs, the Lean files with their compile logs and axiom audits, every file each agent wrote, and the verbatim end-of-attempt interviews; the

per-run readings are given in the supplementary material. The game environments are third party; official replays are linked rather than redistributed.

Statement on AI use

The agents studied in this paper are LLM agents, so AI use is part of the method. Generative AI was also used to draft and edit the text and to write the analysis scripts. All reported numbers come from script outputs over the recorded runs, not from model-generated text. The Lean proofs are checked by the Lean kernel, independently of any model.

References

Tudor Achim et al. Aristotle: IMO-level automated theorem proving. *arXiv preprint arXiv:2510.01346*, 2025.

Javier Aguilar Martín. When a verified world model still loses: Play-adequacy vs prediction-accuracy in LLM-synthesized code world models. *arXiv preprint arXiv:2607.14169*, 2026.

Zergham Ahmed, Kazuki Irie, Joshua B. Tenenbaum, Christopher J. Bates, and Samuel J. Gershman. Learning abstractions for hierarchical planning in program-synthesis agents. *arXiv preprint arXiv:2602.00929*, 2026.

Diego Aineto and Enrico Scala. Action model learning with guarantees. In *Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pp. 801–811, 2024. doi: 10.24963/kr.2024/75.

ARC Prize Foundation. GPT-6 Astra: ARC-AGI results. `https://arcprize.org/results/openai-gpt-6-astra`, 2026a. Public Demo, 25 environments; accessed 2 October 2026.

ARC Prize Foundation. ARC-AGI-3: A new challenge for frontier agentic intelligence. *arXiv preprint arXiv:2603.24621*, 2026b.

Luoxin Chen, Jinming Gu, Liankai Huang, Wenhao Huang, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Kaijing Ma, et al. Seed-Prover: Deep and broad reasoning for automated theorem proving. *arXiv preprint arXiv:2507.23726*, 2025.

Edmund M. Clarke, Orna Grumberg, and David E. Long. Model checking and abstraction. *ACM Transactions on Programming Languages and Systems*, 16(5):1512–1542, 1994. doi: 10.1145/186025.186051.

David Courtis, Wenhao Li, and Scott Sanner. OPINE-World: Programmatic world modeling with ontology-error-prioritized interactive exploration for ARC-AGI-3. *arXiv preprint arXiv:2607.01531*, 2026.

Nicola Dainese, Matteo Merler, Minttu Alakuijala, and Pekka Marttinen. Generating code world models with large language models guided by Monte Carlo tree search. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, volume 37, pp. 60429–60474, 2024. doi: 10.52202/079017-1933.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pp. 625–635. Springer, 2021.

Salomé Eriksson, Gabriele Röger, and Malte Helmert. Unsolvability certificates for classical planning. In *Proceedings of the 27th International Conference on Automated Planning and Scheduling (ICAPS)*, pp. 88–97, 2017. doi: 10.1609/icaps.v27i1.13818.

Leonhard Euler. Solutio problematis ad geometriam situs pertinentis. *Commentarii academiae scientiarum Petropolitanae*, 8:128–140, 1741. Volume dated 1736; Eneström index E53.

Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z. Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, Yannick Schroecker, Hussain Masoom, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106):607–613, 2026. doi: 10.1038/s41586-025-09833-y.

Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. doi: 10.1016/S0004-3702(98)00023-X.

Masaki Kashiwara and Pierre Schapira. *Categories and Sheaves*, volume 332 of *Grundlehren der mathematischen Wissenschaften*. Springer, Berlin, Heidelberg, 2006.

Jens Lehmann, Andrei Aioanei, and Sahar Vahdati. Tycho: Active abstraction with programmatic world models for ARC-AGI-3. *arXiv preprint arXiv:2607.28287*, 2026.

Saunders Mac Lane. *Categories for the Working Mathematician*, volume 5 of *Graduate Texts in Mathematics*. Springer, 2nd edition, 1998.

Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. Certifying algorithms. *Computer Science Review*, 5(2):119–161, 2011. doi: 10.1016/j.cosrev.2010.09.009.

Wasu Top Piriyakulkij, Yichao Liang, Hao Tang, Adrian Weller, Marta Kryven, and Kevin Ellis. PoE-World: Compositional world modeling with products of programmatic experts. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, volume 38, 2025. doi: 10.52202/085713-0896.

Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Z. F. Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.

Sergey Rodionov. Executable world models for ARC-AGI-3 in the era of coding agents. In *Artificial General Intelligence: 19th International Conference, AGI 2026*, Lecture Notes in Computer Science. Springer, 2026a. arXiv:2605.05138.

Sergey Rodionov. Do coding agents need executable world models, simplification, and verification to solve ARC-AGI-3? *arXiv preprint arXiv:2607.15439*, 2026b.

Alexy Skoutnev, Kirill Acharya, Gaston Longhitano, Madeleine Udell, Kevin Ellis, and Iddo Drori. Twin: Playing an unknown game with a test-time digital twin. *arXiv preprint arXiv:2608.14490*, 2026.

Nat Sothanaphan. Resolution of Erdős problem #728: a writeup of Aristotle's Lean proof. *arXiv preprint arXiv:2601.07421*, 2026.

Hao Tang, Darren Key, and Kevin Ellis. WorldCoder, a model-based LLM agent: Building world models by writing code and interacting with the environment. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, volume 37, pp. 70148–70212, 2024. doi: 10.52202/079017-2243.

Pedro A. Tsividis, Joao Loula, Jake Burga, Nathan Foss, Andres Campero, Thomas Pouncy, Samuel J. Gershman, and Joshua B. Tenenbaum. Human-level reinforcement learning through theory-based modeling, exploration, and planning. *arXiv preprint arXiv:2107.12544*, 2021.

George Tsoukalas et al. Advancing mathematics research with AI-driven formal proof search. *arXiv preprint arXiv:2605.22763*, 2026.

David Wang and Mohammad Abdulaziz. Formally verified certification of unsolvability of temporal planning problems. *arXiv preprint arXiv:2510.10189*, 2025.

Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, volume 35, 2022.

Yang Yuan. Succinct representations for concepts. *arXiv preprint arXiv:2303.00446*, 2023.

Haoyu Zhao, Zhengxu Yu, Zhiyuan He, Meng Fang, Rasul Tutunov, Haitham Bou-Ammar, Weilin Luo, and Jun Wang. Memento 3: Model-based recursive self-improvement through reflective rulebooks. *arXiv preprint arXiv:2610.11794*, 2026.